

PROGRAMAÇÃO ORIENTADA A OBJETOS

Prof. André Backes

Programação procedimental ou procedural

- Conjunto de passos computacionais a serem executados
- Problemas são decompostos em sub-problemas
 - Modularização: programa construído usando funções
 - Uma função pode ser chamada a qualquer momento durante a execução do programa
- A ênfase está nas operações desenvolvidas

Programação procedimental ou procedural

- Desvantagens
 - Os dados são separados das funções
 - Dados podem ser lidos e modificados por qualquer função, desordenadamente
 - Mudanças na estrutura dos dados acarreta alteração em todas as funções relacionadas
 - Não existe um modelo de dados
 - A construção de um modelo de dados posteriormente é mais confusa
 - Maior dificuldade na reutilização de código, comumente levando à cópia-cola
 - Gera sistemas difíceis de serem mantidos

Programação orientada a objetos

- Paradigma de programação que se baseia nos conceitos de objetos
 - Entidades que possuem um estado e comportamento definidos
 - Decompõe um problema em objetos que interagem entre si, sendo cada objeto uma unidade de software

Programação orientada a objetos

- Os dados e as funções são vistos de forma agregada.
 - Existe um modelo de dados inerente ao programa
 - A semântica do mundo real é totalmente retratada na solução computacional OO

Programação procedimental x POO

Programação procedimental

- Divide o código em funções ou procedimentos
- Ênfase do desenvolvimento está nas operações desenvolvidas
 - Cada função é responsável por realizar uma tarefa específica
- O objetivo é aumentar a legibilidade e manutenção do código, tornando-o mais organizado e fácil de entender

POO

- Se baseia no conceito de objetos
 - Entidades que possuem estado e comportamento.
- Ênfase está na criação de unidades de software (objetos)
 - Cada unidade contém seus dados e operações possíveis
 - Interação entre elas

Programação orientada a objetos

Vantagens

- Abstração
 - Permite que os objetos sejam representados de forma mais abstrata
- Segurança
 - Permite ocultar detalhes de implementação e expor apenas os métodos e atributos necessários para o funcionamento do sistema
- Reutilização de código
 - Por meio de herança, por exemplo, evitando a duplicação de código

Desvantagens

- Complexidade
 - Envolve o uso de conceitos mais avançados (como herança e polimorfismo) que a programação estruturada.
 - Isso pode dificultar o aprendizado inicial

Programação orientada a objetos

- Principais conceitos da POO são
 - Classe
 - Objeto
 - Encapsulamento
 - Herança
 - Polimorfismo

Classe

- Estrutura de código que atua como um modelo ou um plano para criar objetos
- Descreve
 - propriedades (atributos)
 - comportamentos (métodos)
 - a maneira como esses elementos podem ser acessados (interface) pelo objeto

```
class nome_classe{  
    /// atributos e métodos  
};
```

Classe

- A classe é um molde para a criação de objetos
 - É como a planta arquitetônica que serve de modelo para a construção de uma casa
 - São a base para criar objetos que compartilham características comuns
 - Por exemplo, uma classe para representar pessoas em um sistema

```
class Pessoa{  
    public:  
    string nome;  
    int idade;  
    void saudacao(){  
        cout << "Nome: " << nome << endl;  
        cout << "Idade: " << idade << " anos\n";  
    }  
};
```

Objeto

- É uma instância da classe
 - Ou uma “variável” da classe
 - Possui seus próprios valores de atributos e a capacidade de executar os métodos definidos na classe
 - Seu estado é mantido por meio de atributos
 - Os métodos define o comportamento dos objetos

```
class Pessoa{
    public:
    string nome;
    int idade;
    void saudacao() {
        cout << "Nome: " << nome << endl;
        cout << "Idade: " << idade << " anos\n";
    }
};

int main() {
    Pessoa pe{"Ricardo", 18};
    pe.saudacao();
    //Atualiza a idade
    pe.idade = 19;
    pe.saudacao();

    return 0;
}
```

Objeto

- Possui identidade própria
 - Representação concreta de uma classe
 - É como a casa construída com base em uma planta arquitetônica
- Distinguível de qualquer outro objeto
 - Mesmo que seus atributos sejam idênticos
 - Numa classe de seres humanos, cada pessoa é única

```
class Pessoa{
    public:
    string nome;
    int idade;
    void saudacao() {
        cout << "Nome: " << nome << endl;
        cout << "Idade: " << idade << " anos\n";
    }
};

int main() {
    Pessoa pe{"Ricardo", 18};
    pe.saudacao();
    //Atualiza a idade
    pe.idade = 19;
    pe.saudacao();

    return 0;
}
```

Atributos

- São variáveis associadas a uma classe/ objeto
 - Conjunto de características do objeto
 - Definem as propriedades que queremos para o nosso objeto
 - Valores internos do objeto
 - Ponto: coordenadas
 - Seres humanos: nome, peso, altura, ...
 - São acessados usando o operador ponto (.)

```
class Pessoa{
    public:
        string nome;
        int idade;
        void saudacao() {
            cout << "Nome: " << nome << endl;
            cout << "Idade: " << idade << " anos\n";
        }
};

int main() {
    Pessoa pe{"Ricardo", 18};
    pe.saudacao();
    //Atualiza a idade
    pe.idade = 19;
    pe.saudacao();

    return 0;
}
```

Métodos

- Conjunto de funcionalidades da classe
 - Funções definidas dentro de uma classe
 - São usados para realizar operações ou manipulações nos atributos do objeto
 - Definem o comportamento dos objetos criados a partir dessa classe
 - Ponto: acessar X e Y, calcular distância, ...
 - Seres humanos: correr, nadar, ...
 - São acessados usando o operador ponto (.)

```
class Pessoa{
    public:
    string nome;
    int idade;
    void saudacao() {
        cout << "Nome: " << nome << endl;
        cout << "Idade: " << idade << " anos\n";
    }
};

int main() {
    Pessoa pe{"Ricardo",18};
    pe.saudacao();
    //Atualiza a idade
    pe.idade = 19;
    pe.saudacao();

    return 0;
}
```

Métodos

- Podem ser implementados fora da classe
 - Protótipo fica dentro da classe
 - Na implementação é necessário colocar o identificador da classe e o operador de resolução de escopo :: antes do nome do método

```
class Pessoa{  
    public:  
    string nome;  
    int idade;  
    void saudacao();  
};  
// método implementado fora da classe  
void Pessoa::saudacao(){  
    cout << "Nome: " << nome << endl;  
    cout << "Idade: " << idade << " anos\n";  
}
```

Método

- Assim como as funções tradicionais, os métodos de uma classe também podem ser sobrecarregados
 - Permite criar várias funções com o mesmo nome, desde que elas tenham parâmetros diferentes

```
class Pessoa{
public:
    string nome;
    int idade;
    void saudacao(){
        cout << "Nome: " << nome << endl;
        cout << "Idade: " << idade << " anos\n";
    }
    void saudacao(string texto){
        cout << texto << endl;
        cout << "Nome: " << nome << endl;
        cout << "Idade: " << idade << " anos\n";
    }
};

int main(){
    Pessoa pe{"Ricardo",18};
    pe.saudacao();
    pe.saudacao("Saudacao personalizada!");

    return 0;
}
```


Classe vs Estrutura

- Os membros de uma **struct** são **públicos**
 - Podem ser acessados diretamente

```
struct cadastro{  
    string nome;  
    int idade;  
    string rua;  
    int numero;  
};  
  
int main(){  
    cadastro cad;  
  
    cad.idade = 18;  
    cout << cad.idade;  
    return 0;  
}
```

Classe vs Estrutura

- Os membros de uma **classe** são, por padrão, **privados**
 - Só podem ser acessados por meio de métodos públicos
 - Podemos especificar essa visibilidade

```
class cadastro{  
    string nome;  
    int idade;  
    string rua;  
    int numero;  
};  
  
int main() {  
    cadastro cad;  
    //Erro ao acessar idade  
    cad.idade = 18;  
    cout << cad.idade;  
    return 0;  
}
```

Classe vs Estrutura

- struct
 - Usada para agrupar dados de forma mais simples
- Classe
 - Usada para criar objetos com comportamento mais complexo e encapsulamento de dados
 - Mais recursos
 - Construtor
 - Destrutor
 - Métodos virtuais
 - etc.

Modificadores de acesso

- Em várias situações, o acesso direto aos atributos de um objeto não é aconselhável
 - Pode comprometer a lógica e funcionamento da classe
 - Altera dados do objeto sem verificação apropriada
- Algumas linguagens permitem restringir o acesso aos atributos

Modificadores de acesso

- Os modificadores controlam a visibilidade e acessibilidade de atributos e métodos
 - Conceito fundamental da programação orientada a objetos
 - Eles determinam a visibilidade e acessibilidade fora da classe
 - Fica exposto apenas os métodos e atributos necessários para o funcionamento do sistema
 - Promove o encapsulamento e a segurança dos dados

Modificadores de acesso

- Restringem ou permitem o acesso a atributos e métodos
 - Garantindo que apenas partes autorizadas do código possam interagir com determinados elementos da classe
- Existem três tipos principais de modificadores de acesso:
 - **private**
 - **public**
 - **protected**

Modificador **public**

- Permite o acesso direto aos membros da classe
 - Atributos e métodos podem ser acessados de qualquer parte do programa
 - Isso inclui funções externas e outras classes

```
class Pessoa {  
    public:  
    string nome;  
  
    void definirNome(string n) {  
        nome = n;  
    }  
};  
  
int main() {  
    Pessoa p;  
    p.definirNome("Carlos");  
    cout << p.nome << endl;  
  
    return 0;  
}
```

Modificador **private**

- Proíbe o acesso direto aos membros da classe
 - Atributos e métodos só podem ser acessados pelos métodos da própria classe
 - Proíbe o acesso por funções externas e outras classes
 - É o **modo padrão**
 - A classe assume que todos os atributos e métodos são privados se nenhum modificador de acesso for especificado

```
class ContaBancaria {  
    private:  
        double saldo;  
  
    public:  
        void definirSaldo(double s) {  
            saldo = s;  
        }  
        double obterSaldo() {  
            return saldo;  
        }  
};  
  
int main() {  
    ContaBancaria conta;  
    conta.definirSaldo(1000.50);  
    cout << "Saldo: " << conta.obterSaldo() << endl;  
  
    // Erro de compilação  
    cout << conta.saldo << endl;  
  
    return 0;  
}
```


Modificador **protected**

- Proíbe o acesso direto, mas permite o acesso por classes derivadas
 - Os atributos e métodos são privados, mas podem ser acessados por classes derivadas (herança)
 - Podem ser acessados diretamente pela própria classe e pelas classes filhas

```
class Veiculo{
    protected:
        int velocidade;

    public:
        void definirVelocidade(int v) {
            velocidade = v;
        }
};

class Carro: public Veiculo{
public:
    void acelerar() {
        velocidade += 10;
    }
};

int main() {
    Carro c;
    c.definirVelocidade(50);
    c.acelerar();

    // Erro de compilação
    cout << c.velocidade;

    return 0;
}
```

Comparação entre os modificadores

Modificador	Acessível Dentro da Classe	Acessível por Classes Derivadas	Acessível Fora da Classe
public	Sim	Sim	Sim
protected	Sim	Sim	Não
private	Sim	Não	Não

Coesão e acoplamento

- Conceitos importantes na POO
 - Eles se referem à forma como os elementos de um sistema de software (como as classes, módulos, objetos, etc.) estão relacionados uns com os outros
 - De modo geral, é importante manter um alto grau de coesão e um baixo grau de acoplamento
 - Isso tornar o sistema mais fácil de entender e manter

Coesão e acoplamento

- Coesão

- Se refere à medida em que um elemento de um sistema de software (como uma classe) é focado em uma única tarefa ou responsabilidade
- Um elemento de alto grau de coesão é focado em uma única tarefa e possui um conjunto de responsabilidades claramente definidas
- Isso torna o elemento mais fácil de entender e dar manutenção

Coesão e acoplamento

- ContaBancaria possui uma alta coesão
 - Todos os métodos estão focados em tarefas relacionadas à conta bancária
 - Como depositar, sacar e consultar o saldo
 - Fácil de entender e manter
 - Seus métodos têm um conjunto de responsabilidades claramente definidas

```
class ContaBancaria{
    private:
        double saldo = 0.0;

    public:
        void depositar(double valor){
            saldo += valor;
        }

        void sacar(double valor) {
            if(valor > saldo)
                cout << "Saldo insuficiente!" << endl;
            else
                saldo -= valor;
        }

        double consultarSaldo(){
            return saldo;
        }
};
```

Coesão e acoplamento

- Acoplamento

- Se refere à medida em que um elemento de um sistema de software (como uma classe) é dependente de outro elemento
- Quanto mais alto o grau de acoplamento, mais dependente um elemento é de outro elemento
 - Isso pode tornar o sistema mais difícil de entender e dar manutenção
 - Alterações em um elemento podem afetar outros elementos do sistema

Coesão e acoplamento

- Empresa está acoplada à classe ContaBancaria
 - Possui um atributo privado que é uma referência a ContaBancaria
 - Depende dela para realizar as suas tarefas
 - Alterações em ContaBancaria podem afetar o funcionamento da classe Empresa

```
class Empresa{  
    private:  
        ContaBancaria& conta;  
  
    public:  
        Empresa(ContaBancaria& conta_bancaria): conta(conta_bancaria){}  
  
        void pagarSalarios(double valor) {  
            conta.sacar(valor);  
        }  
  
        void realizarCompra(double valor) {  
            conta.sacar(valor);  
        }  
};
```

Material Complementar

- Aula 23 - POO: Classe e Objeto
 - <https://youtu.be/lfaxWRivc5E>
- Aula 24 - POO: Modificadores de Acesso
 - <https://youtu.be/nrKED68blqs>